

Wheel Candidate Annotator User Manual

Project: Lightweight Wheel-Candidate Filtering for Virtual Wheel Try-On

Audience: team members who annotate candidate boxes

Level: no machine learning experience required

1. Goal of the Tool

Wheel Candidate Annotator is a PyQt application for preparing a dataset for a binary classification task.

The model will learn to answer one question:

`Is this specific candidate box a good wheel candidate?`

It will not answer:

`Does the whole image contain a wheel?`

This difference is important. You must label each candidate rectangle, not the full image.

The labels are:

- `valid_wheel_candidate` - label 1, used as a positive training example;
- `invalid_candidate` - label 0, used as a negative training example;
- `ignore` - saved in JSON, but not used for model training.

Simple rule:

- Use `valid_wheel_candidate` if this box should continue to the wheel try-on pipeline.
- Use `invalid_candidate` if this box clearly should not continue.
- Use `ignore` if you are not sure after a few seconds.

2. Important Annotation Rule

If a box was produced by the candidate generator, usually do not fix it to make it perfect.

Label the candidate box as it is.

This is important because the future model must work on real candidate boxes from the pipeline.

If we manually improve all bad boxes, the dataset will become less realistic.

Create a new manual box only when the task explicitly asks you to add new examples.

3. Start the Application

Open PowerShell and run:

```
cd C:\Users\user\Downloads\transformers\ass2
python .\wheel_candidate_annotator.py
```

If the project is in a different folder, replace the path with your project path.

If you see an error about missing PyQt6 or Pillow, run:

```
python -m pip install PyQt6 pillow
```

Then start the application again.

4. Main Folders

Purpose	Path
Input images by default	C:\Users\user\Downloads\transformers\ass2\datasets_ra
JSON annotations	C:\Users\user\Downloads\transformers\ass2\wheel_cand
Exported crop images	C:\Users\user\Downloads\transformers\ass2\wheel_cand

If your project folder is different, the beginning of these paths will also be different.

5. Interface Structure

The interface has three main areas:

1. Left panel: dataset list, search, filters, and annotation progress.
2. Center area: image with candidate boxes.
3. Right panel: image fields and selected box fields.

The full window looks like this:



Figure 1: Annotator main window with candidate boxes

The right panel is where you set the label for the selected box:

6. Visual Examples

Use these examples as a guide. The exact image may be different in the tool, but the decision logic is the same.

20210405_18_10_19_000_2sQ7uoDSHQakPcTTKy9lkfs5wWs1_F_326-
2448 x 3264 px
5 boxes | valid 5 | invalid 0 | ignore 0

Image-level fields

Split
unassigned

Vehicle type
6_wheeler

Viewpoint
unknown

Image quality
good

Image source
real

Notes

Selected box / new box defaults

Object
wheel

Candidate label
valid_wheel_candidate

Wheel position
front

Side
left

Visibility
full

Crop quality
good

Box source
manual

Box notes

Mark valid

Mark invalid

#	label	object	pos	side	vis	
1	1	wheel	unknown	unknown	full	g
2	1	wheel	unknown	unknown	unknown	g
3	1	wheel	unknown	unknown	unknown	g
4	1	wheel	unknown	unknown	unknown	g

Delete box

Import XML

Save JSON

Export crops

Crop margin
15%

Output
C:\Users\user\Downloads\transformers\ass2\wheel_

Export all labeled crops

Figure 2: Selected box panel with valid and invalid controls

6.1 Valid Wheel Candidate



Figure 3: Valid wheel candidate example

Recommended label: `valid_wheel_candidate`.

Reason: the crop contains a clear physical wheel. It has enough tire and rim information. This type of candidate is useful for the next stage of the wheel try-on pipeline.

6.2 Invalid Candidate: Headlight



Figure 4: Invalid headlight example

Recommended label: `invalid_candidate`.

Reason: a headlight can be round and visually similar to a wheel, but it is not a wheel. This is a useful hard negative example.

6.3 Invalid Candidate: Rain and Pavement Reflection



Figure 5: Rain and pavement reflection example

Recommended label for a reflected or wet-road region: `invalid_candidate`.

Reason: reflections and wet pavement can create wheel-like shapes. A reflected shape is not a physical wheel and should not be passed to the try-on stage.

6.4 Invalid Candidate: Light Reflection in a Puddle

Recommended label for the reflected region: `invalid_candidate`.

Reason: the reflected light is not a real wheel. This example shows why the model needs hard negative samples, not only random background.

6.5 Source and License Notes for External Examples

The external example images in this manual are from Wikimedia Commons:

Manual image	Source	License
Valid wheel	Wheel of car.jpg, author Iorisrandombses5001	CC0 1.0
Headlight	330ci xenon.jpg, author Dr.jameshughes	Public domain
Car in rain	Car in rain, Seattle, 1980s.jpg, author Seattle Municipal Archives	CC BY 2.0
Light reflection	Tail light reflection in Tuntorp.jpg, author W.carter	CC BY-SA 4.0



Figure 6: Light reflection in puddle example

The images are used only as annotation examples. When possible, keep the original source link and license information together with the manual.

7. Fast Workflow

1. Start the application.
2. Check that the correct image folder is open.
3. If there is a Pascal VOC .xml file near the image, the application imports wheel boxes automatically.
4. Click a candidate box.
5. Label the selected box as `valid_wheel_candidate`, `invalid_candidate`, or `ignore`.
6. Add optional metadata only if it is easy and clear.
7. Move to the next image.
8. After annotation, click **Export all labeled crops**.
9. Check that the export folder contains `valid_wheel_candidate/`, `invalid_candidate/`, and `labels.csv`.

8. Mouse Controls

Action	Result
Left mouse button + drag on image	Draw a new box.
Left mouse click on a box	Select a box for editing.
Mouse wheel	Zoom in or zoom out.
Right mouse button + drag	Move the image.
Middle mouse button + drag	Move the image.

Use manual drawing only when needed. For the main dataset, imported or generated candidate boxes should usually be labeled as they are.

9. Keyboard Shortcuts

Key	Action
1	Mark selected box as <code>valid_wheel_candidate</code> .
0	Mark selected box as <code>invalid_candidate</code> .
Del	Delete selected box.
N	Go to next image.
P	Go to previous image.
F	Fit image to window.
Ctrl+S	Save JSON annotation.

10. Left Panel: Dataset

Buttons:

- `Open folder` - open another image folder.
- `Reload` - reload the current folder.
- `Next not annotated` - go to the next image without JSON annotation.
- `Previous` and `Next` - move between images.

Search and filters:

- `Search file name` filters the list by file name.
- `all images` shows all images.
- `not annotated` shows images without JSON annotation.
- `has boxes` shows images with boxes.
- `no boxes` shows images without boxes.

In the list:

- `DONE` means that a JSON file already exists for the image.
- `TODO` means that no JSON file exists yet.

11. Right Panel: Image-Level Fields

These fields describe the whole image, not one box.

Field	How to use
Split	<code>train</code> , <code>validation</code> , <code>test</code> , or <code>unassigned</code> . It is fine to leave <code>unassigned</code> if split will be created later by script.
Vehicle type	Example values: <code>car</code> , <code>suv</code> , <code>truck</code> , <code>bus</code> , <code>motorcycle</code> , <code>2_wheeler</code> , <code>3_wheeler</code> .
Viewpoint	Example values: <code>side</code> , <code>front_3_4</code> , <code>rear_3_4</code> , <code>front</code> , <code>rear</code> , <code>unknown</code> .
Image quality	Example values: <code>good</code> , <code>low_resolution</code> , <code>blurry</code> , <code>occluded</code> , <code>dark</code> .
Image source	Example values: <code>real</code> , <code>generated</code> , <code>mixed</code> , <code>unknown</code> .

Field	How to use
Notes	Free text comment about the whole image.

12. Right Panel: Selected Box Fields

These fields describe the selected box. If no box is selected, they may define default values for a new box.

12.1 Object

Value	Meaning
wheel	A real visible wheel.
wheel_candidate	A region that looks like a possible wheel.
non_wheel_candidate	A region that may be proposed by the system, but is not a wheel.
wheel_arch	Wheel arch without a useful wheel.
body_panel	Car body.
headlight	Headlight.
road_background	Road or background.
generated_artifact	Generation or overlay artifact.
other	Other object.

12.2 Candidate Label

Value	Meaning
valid_wheel_candidate	Good wheel crop. It will be exported as label 1.
invalid_candidate	Bad or false candidate. It will be exported as label 0.
ignore	Saved in JSON, but not used in training export.

12.3 Other Box Fields

Field	Values and meaning
Wheel position	front, rear, middle, spare, unknown, not_applicable.
Side	left, right, center, unknown, not_applicable.
Visibility	full, partial, occluded, truncated, unknown.
Crop quality	good, slightly_blurry, distorted, bad_overlay, too_small.
Box source	manual, imported_xml, generated, auto_negative.
Box notes	Free text comment about the selected box.

Field	Values and meaning
-------	--------------------

13. How to Label Valid Examples

Use `valid_wheel_candidate` if:

- the box contains a real wheel;
- the wheel is visible enough;
- the crop is useful for the next pipeline stage;
- the box does not include too much irrelevant background;
- the box does not cut off the wheel in a critical way;
- the crop contains a little useful context, such as tire, wheel arch, or car body.

A good box does not need to be a perfect circle around the rim. A small amount of context is useful for training.

14. How to Label Invalid Examples

Use `invalid_candidate` if the box clearly contains:

- a headlight;
- a wheel arch without a useful wheel;
- car body;
- road or background;
- bumper or license plate;
- a shadow;
- a reflection that is not a physical wheel;
- a random round object;
- a strongly distorted wheel;
- a wrong wheel overlay;
- a generation artifact where the wheel looks broken.

Negative examples should be similar to real system mistakes. Do not make all negative examples random background. The model needs to learn the difference between a wheel and hard false positives.

15. When to Use Ignore

Use `ignore` if:

- you are not sure about the label;
- the box is very strange;
- the image quality is very poor;
- the wheel is almost invisible;
- the example may confuse the model;
- you do not want this sample to be used for training.

Practical rule: if you think for more than a few seconds and still feel unsure, use `ignore`.

16. Examples of Common Decisions

Case	Label
Clear front or rear wheel, box is reasonable	<code>valid_wheel_candidate</code>
Wheel is slightly cropped, but still useful	<code>valid_wheel_candidate</code>
Box contains only wheel arch and body	<code>invalid_candidate</code>
Box contains a reflection of a wheel on wet road	<code>invalid_candidate</code>
Box contains a headlight or round lamp	<code>invalid_candidate</code>
Box contains a background vehicle wheel, not the target car	Usually <code>invalid_candidate</code>
Box is too confusing to decide	<code>ignore</code>

17. Saving JSON

The application saves JSON automatically:

- when fields change;
- when you move to another image;
- when you close the application.

You can also click **Save JSON** or press **Ctrl+S**.

JSON files are saved in:

`C:\Users\user\Downloads\transformers\ass2\wheel_candidate_dataset\annotations_json`

Each JSON contains:

- path to the source image;
- image size;
- image-level metadata;
- list of boxes;
- object label;
- candidate label;
- binary label;
- box coordinates;
- quality and visibility fields.

18. Export Crop Dataset

When annotation is ready, click:

Export all labeled crops

The application will create:

`C:\Users\user\Downloads\transformers\ass2\wheel_candidate_dataset\labeled_crops`

Inside this folder, you should see:

- `valid_wheel_candidate/;`
- `invalid_candidate/;`
- `labels.csv`.

The `labels.csv` file contains:

- crop path;
- label;
- source image;
- source box;
- crop box with margin;
- object label;
- wheel position;
- side;
- visibility;
- crop quality;
- split;
- vehicle type;
- viewpoint.

19. Recommended Annotation Volume

For the first working version:

300 valid + 300 invalid

For a stronger course result:

800-1200 valid + 800-1200 invalid

Priority for invalid examples:

1. wheel arches without the wheel;
2. reflections and shadows;
3. headlights and round car parts;
4. body parts near the wheel;
5. road and background near the vehicle;
6. bad generated or overlay candidates.

20. Final Checklist Before Sending the Dataset

Before you send your annotation result, check:

- the correct image folder was used;
- the JSON annotation folder exists;
- most images are marked DONE;
- `Export all labeled crops` was clicked;
- the export folder contains `valid_wheel_candidate/`;
- the export folder contains `invalid_candidate/`;
- the export folder contains `labels.csv`;
- unclear samples were marked as `ignore`, not guessed.

21. Short Rule for Annotators

Do not label the full image.

Label the selected rectangle:

```
this box = useful wheel candidate  
or  
this box = not useful wheel candidate  
If unsure, use ignore.
```